

Lab 10: Spring WebFlux & WebClient

1. Reactive Programming vs Blocking ต่างกันอย่างไร

Blocking I/O คือการทำงานที่ Thread ต้องรอผลลัพธ์จากการทำงานก่อน จึงจะสามารถทำงานต่อได้ ถ้ามีหลาย Request อาจทำให้ต้องใช้ Thread จำนวนมากในการรอ (Request → รอผลลัพธ์ → ทำงานต่อ)

Reactive Programming คือการเขียนโปรแกรมแบบ Asynchronous ที่สามารถทำงานกับ Data Stream โดยไม่ต้องให้ Thread รอผลลัพธ์ ทำให้สามารถนำ Thread ไปทำงานอื่นระหว่างรอได้ (Request → ส่งงาน →ทำงานอื่น →รับผลลัพธ์เมื่อพร้อม)

2. Mono vs Flux ใช้กรณีไหน และทำไม

Mono<T> ใช้สำหรับข้อมูลที่มี 0 หรือ 1 ค่า เช่น การค้นหา Product จาก ID หรือการบันทึก Product

Flux<T> ใช้สำหรับข้อมูลที่มี 0 ถึงหลายค่า เช่น การดึง Product ทั้งหมด หรือการค้นหา Product ตาม Category

ใน Lab นี้มีการใช้งานดังนี้

Java

GET /products/{id} → Mono<Product>

ใช้ **Mono** เพราะการค้นหาด้วย ID จะได้ Product เพียง **1** รายการ หรืออาจไม่พบข้อมูลเลย

POST /products → Mono<Product>

ใช้ **Mono** เพราะหลังจากเพิ่ม Product ระบบจะส่ง **Product** ที่สร้างขึ้นกลับมาเพียง **1** รายการ

DELETE /products/{id} → Mono<Void>

ใช้ **Mono<Void>** เพราะการลบ Product ไม่จำเป็นต้องส่งข้อมูล Product กลับมา เพียงแค่ทำงานให้เสร็จแล้วจบการทำงาน

GET /products → Flux<Product>

ใช้ **Flux** เพราะ Endpoint นี้ต้องดึง Product หลายรายการ กลับมา เช่น Product ทั้งหมดในระบบ

GET /products/category/{category} → Flux<Product>

ใช้ **Flux** เพราะ Category หนึ่งสามารถมี Product ได้ หลายรายการ เช่น Electronics มีทั้ง iPhone, MacBook และ Samsung

GET /products/{id}/price → Mono<Double>

ใช้ **Mono** เพราะการค้นหาราคาของ Product จาก ID จะได้ ราคากลับมาเพียง **1** ค่า เช่น **35910.0**

3. WebClient อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

WebClient ใช้สำหรับเรียก REST API แบบ Reactive โดยใน Lab นี้ใช้ WebClient เชื่อมต่อกับ Server ที่ localhost:8080

ตัวอย่างการเรียกข้อมูล Product ทั้งหมด

Java

```
client.get()
    .uri("/products")
    .retrieve()
    .bodyToFlux(Product.class);
```

คำอธิบายแต่ละ Method

- .get() ใช้กำหนด HTTP Method เป็น GET
- .uri() ใช้กำหนด URL ที่ต้องการเรียก
- .retrieve() ใช้รับ Response จาก Server
- .bodyToFlux(Product.class) ใช้แปลง Response เป็น Flux<Product>

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

4.1 ProductRepository

ทำหน้าที่จัดเก็บและจัดการข้อมูลสินค้า โดยใน Lab นี้ใช้ ConcurrentHashMap เป็นฐานข้อมูลแบบ In-Memory

Java

```
public Mono<Product> findById(String id) {
    return Mono.justOrEmpty(products.get(id));
}

public Flux<Product> findAll() {
    return Flux.fromIterable(products.values());
}

public Mono<Product> save(Product product) {
    products.put(product.getId(), product);
    return Mono.just(product);
}

public Mono<Void> deleteById(String id) {
    products.remove(id);
    return Mono.empty();
}

public Flux<Product> findByCategory(String category) {
    return Flux.fromIterable(products.values())
        .filter(p ->
            p.getCategory().equalsIgnoreCase(category));
}
```

Repository เป็นส่วนที่จัดการข้อมูลสินค้า เช่น ค้นหา เพิ่ม ลบ และค้นหาตามหมวดหมู่ โดยใช้ **Mono** สำหรับข้อมูล 1 รายการ และ **Flux** สำหรับข้อมูลหลายรายการ

4.2 ProductService

ทำหน้าที่เป็นตัวกลางระหว่าง Controller และ Repository และจัดการ Logic ของโปรแกรม

Java

```
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(new RuntimeException("Product not found:
" + id))
        );
}

public Flux<Product> getAll() {
    return repository.findAll();
}

public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }
    return repository.save(product);
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(Product::getDiscountedPrice);
}
```

Service เป็นส่วนที่จัดการ Logic เช่น การสร้าง ID ใหม่เมื่อเพิ่มสินค้า และการคำนวณราคาหลังหักส่วนลด โดยใช้ map() และ switchIfEmpty() ในการจัดการข้อมูลแบบ Reactive

4.3 ProductController

ทำหน้าที่รับ Request จากผู้ใช้และส่งผลลัพธ์กลับเป็น REST API

Java

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

@GetMapping("/category/{category}")
public Flux<Product> getByCategory(
    @PathVariable String category) {
    return service.getByCategory(category);
}

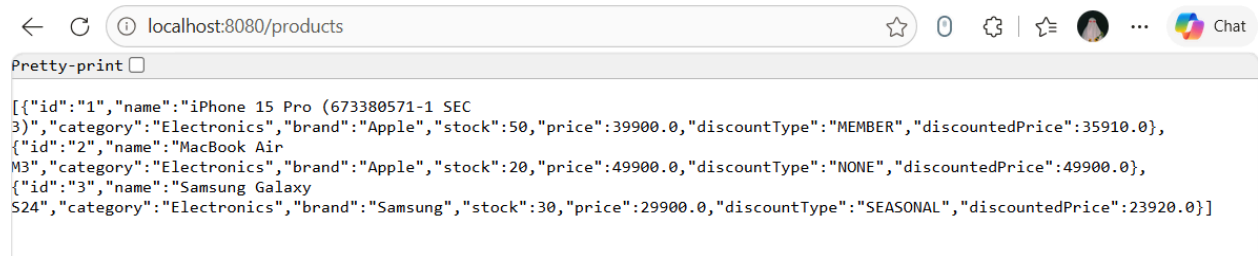
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(
    @PathVariable String id) {
    return service.getDiscountedPrice(id);
}
```

Controller รับ HTTP Request เช่น GET, POST และ DELETE แล้วเรียกใช้ Service เพื่อประมวลผล จากนั้นส่งผลลัพธ์กลับในรูปแบบ **Mono** หรือ **Flux**

สรุปแนวทางการแบ่ง layer: ทั้ง 3 layer นี้ยึดหลัก Single Responsibility Principle (SOLID) Repository จัดการแค่ data storage, Service จัดการ business rule (เช่น throw error ถ้าไม่เจอ, generate id), Controller จัดการแค่รับ-ส่ง HTTP request/response เท่านั้น ทำให้แก้ไข/ทดสอบแต่ละส่วนแยกกันได้โดยไม่กระทบส่วนอื่น

5. ผลทดสอบ Endpoint ทั้ง 6

5.1 ดึง Product ทั้งหมด

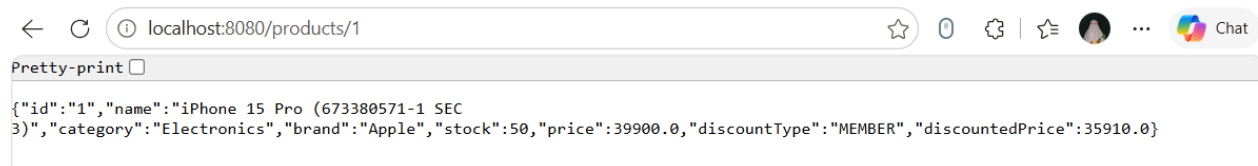


```
localhost:8080/products

Pretty-print

[{"id": "1", "name": "iPhone 15 Pro (673380571-1 SEC 3)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}, {"id": "2", "name": "MacBook Air M3", "category": "Electronics", "brand": "Apple", "stock": 20, "price": 49900.0, "discountType": "NONE", "discountedPrice": 49900.0}, {"id": "3", "name": "Samsung Galaxy S24", "category": "Electronics", "brand": "Samsung", "stock": 30, "price": 29900.0, "discountType": "SEASONAL", "discountedPrice": 23920.0}]
```

5.2 ดึง Product 1 รายการตาม id



```
localhost:8080/products/1

Pretty-print

{"id": "1", "name": "iPhone 15 Pro (673380571-1 SEC 3)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}
```

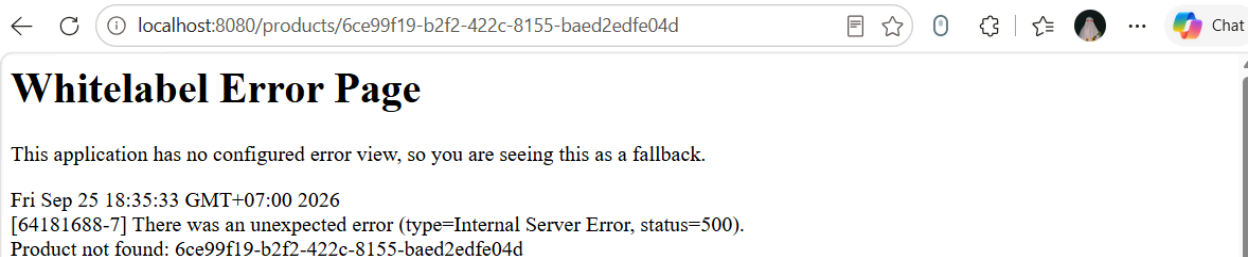
5.3 สร้าง Product ใหม่

```
PS E:\Study\PDA\Lab\lab10-673380571-1-sec3> $body = @{
    >>     name = "Keyboard"
    >>     category = "Electronics"
    >>     brand = "Logitech"
    >>     stock = 10
    >>     price = 1200.0
    >>     discountType = "NONE"
    >> } | ConvertTo-Json
PS E:\Study\PDA\Lab\lab10-673380571-1-sec3>
PS E:\Study\PDA\Lab\lab10-673380571-1-sec3> Invoke-RestMethod `
    >>     -Uri "http://localhost:8080/products" `
    >>     -Method Post `
    >>     -ContentType "application/json" `
    >>     -Body $body

id           : 6ce99f19-b2f2-422c-8155-baed2edfe04d
name          : Keyboard
category      : Electronics
brand         : Logitech
stock         : 10
price         : 1200.0
discountType  : NONE
discountedPrice : 1200.0
```

5.4 ลบ Product ใหม่

```
PS E:\Study\PDA\Lab\lab10-673380571-1-sec3> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/products/6ce99f19-b2f2-422c-8155-baed2edfe04d" `
>>     -Method Delete
```



5.5 กรอง Product ตาม category

```
localhost:8080/products/category/Electronics

Pretty-print ☐

[{"id": "1", "name": "iPhone 15 Pro (673380571-1 SEC 3)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}, {"id": "2", "name": "MacBook Air M3", "category": "Electronics", "brand": "Apple", "stock": 20, "price": 49900.0, "discountType": "NONE", "discountedPrice": 49900.0}, {"id": "3", "name": "Samsung Galaxy S24", "category": "Electronics", "brand": "Samsung", "stock": 30, "price": 29900.0, "discountType": "SEASONAL", "discountedPrice": 23920.0}]
```

5.6 ดูราคา Product

```
localhost:8080/products/1/price

Pretty-print ☐

35910.0
```